Supplementary Appendix for "Diagnosing and Correcting Geometric Bias in Diversity-Driven Code Instruction Data Selection"

1 Data Construction Pipeline

We construct the candidate pool through three steps: source aggregation, response regeneration, and rule-based cleaning. The goal of this pipeline is to obtain a code-focused instruction-response pool with consistent response formatting and reduced invalid outputs.

1.1 Source Aggregation & Filtering

We aggregate instructions from three open-source datasets: *EpiCoder-func-380k*, *Llama-Nemotron-Post-Training*, and *OpenThoughts-114k*. Because these sources differ in domain coverage, we apply source-specific filtering using Qwen3-32B as an instruction-level relevance filter:

- For the larger general-purpose instruction datasets, *EpiCoder-func* and *Llama-Nemotron*, we prompt the model to retain code-related instructions.
- For the reasoning-focused dataset, *OpenThoughts*, we prompt the model to retain LeetCode-style algorithmic tasks.

1.2 Response Regeneration

Different data sources often exhibit varying response styles and quality levels. To mitigate this distribution shift and ensure high-quality supervision, we unify the response distribution. We employ Qwen3-32B to regenerate the code responses for all retained instructions. This step is part of candidate-pool construction and is shared by all selection methods; and it makes the supervision format and coding style more consistent across the candidate pool, reducing noise introduced by heterogeneous data origins while preserving the original instruction side used for selection.

1.3 Quality Cleaning

We then apply rule-based cleaning to remove clearly invalid instruction-response pairs. Specifically, we remove:

(1) samples whose regenerated response does not contain a valid code block, such as responses with Markdown formatting errors or purely textual explanations;
(2) samples with obvious degeneration, such as empty outputs or excessive repetition;
(3) duplicate instruction-response pairs after regeneration and formatting normalization.

After filtering, response regeneration, cleaning, and deduplication, the final candidate pool contains $N = 446{,}062$ instruction-response pairs.

2 Macro-Intent Generation Details

To construct the macro-intent representation, we use an open-ended label generation step followed by label normalization and filtering.

2.1 Prompt Templates

We use Qwen3-8B as the offline label generator for macro-intent label generation. The label-generation prompt contains only the instruction text and does not observe regenerated responses, downstream evaluation data, or test cases. We considered two prompting strategies: a streamlined direct prompt that directly generates descriptive labels from the instruction, and an alternative CoT prompt that first asks LLM to analyze the problem goal, algorithms, data structures, and techniques. Our preliminary experiments indicated that the streamlined direct prompt yields labels of comparable quality to the CoT approach while significantly reducing token consumption. We therefore adopt the streamlined direct prompt for full-scale label generation to reduce cost, and include the alternative CoT prompt for completeness.

2.1.1 *Streamlined Direct Prompt.*

Prompt: Label Generation (Easy)

````
System Prompt:
You are a senior AI Python programming assistant. Your task is
      to analyze a given programming problem to generate a set
      of descriptive labels. These labels should accurately
      capture the core essence of the task.

User Prompt:
Input Provided:
- Problem Description

Your Task:
Analyze the given problem. Generate descriptive labels that
      capture the core essence of the task. Provide the final
      JSON output of labels enclosed in markdown fences (start
      with "```label" and end with "```").

Output Format (Strictly Follow This Order):

```label
{
  "labels": [
    "Generated_Label_1",
    "Generated_Label_2",
    "..."
  ]
}
```
Input: Problem: {problem}
````

2.1.2 *Complex CoT Prompt.*

Complex CoT Prompt

```
System Prompt:
```

```
You are a senior AI Python programming assistant. Your task is
      to analyze a given programming problem to generate a set
      of descriptive labels. These labels should accurately
      capture the core essence of the task, including the
      problem domain, algorithms/data structures used, and key
      programming concepts.

User Prompt:
Input Provided:
- Problem Description

Your Task:
1. Analyze the Input: Carefully read the problem description.
      Understand what the problem is asking and how to use code
       to solve it.
2. Identify Key Characteristics: Deconstruct its fundamental
      components.
   - What is the main goal? (e.g., searching, sorting, data
      processing)
   - What specific algorithms are needed? (e.g., binary search,
       depth-first search, dynamic programming)
   - What primary data structures are involved? (e.g., array,
      hash map, tree, graph)
   - Are there any notable programming paradigms or techniques?
       (e.g., recursion, bit manipulation, object-oriented
      programming)
3. Generate Labels: Based on your analysis, create a list of
      labels that are:
   - Concise: Use short, widely-recognized terms (e.g., "Binary
       Search", not "Searching in a sorted array").
   - Canonical: Use standard, industry-accepted names (e.g., "
      Hash Table" instead of "Dictionary", "Depth-First Search"
       instead of "DFS Traversal").
   - Hierarchical (if applicable): Start with fundamental data
      structures or domains (e.g., "Array", "Graph"), followed
      by more specific algorithms or techniques (e.g., "Two
      Pointers", "Topological Sort").
   - Relevant: Each label must be supported by evidence in the
      code or problem description.

Maintain the original output format. You must write everything
      in your own words, with good structure and reasoning.

Output Format (Strictly Follow This Order):

First, provide a detailed analysis based on the "Identify Key
      Characteristics" step. Second, provide the final JSON
      output of labels enclosed in markdown fences (start with
      "```label" and end with "```").

1. Analysis of Key Characteristics:
   - Main Goal: Analyze the main goal of the problem
   - Algorithms: Analyze the algorithms needed
   - Data Structures: Analyze the data structures needed
   - Techniques: Analyze other techniques needed

2. Generated Labels
```label
{
  "labels": [
    "Generated_Label_1",
    "Generated_Label_2",
    "..."
  ]
}
```
Input: Problem: {problem}
```

2.2 Label Normalization & Filtering Pipeline

Because generation is open-ended rather than restricted to a predefined vocabulary, raw labels may contain surface variations such as capitalization, punctuation, or morphological variants (e.g., "Sorting" vs. "sort"). To obtain a compact and reusable label vocabulary, we apply the following normalization and filtering pipeline:

(1) **Text normalization.** We lowercase all labels and remove non-alphanumeric characters to reduce surface-form variation.
(2) **Canonical mapping.** We apply lemmatization using NLTK's WordNet Lemmatizer to map morphological variants to a shared base form. Labels mapped to the same lemma are grouped, and the most frequent surface form in the corpus is used as the canonical representative.
(3) **Support filtering.** We retain canonical labels whose global frequency is at least $\tau = 5$. This support filter removes one-off or overly specific labels and stabilizes the label vocabulary used to construct macro-intent anchors.

2.3 Label Statistics

Table 1 summarizes the macro-intent label statistics before and after normalization and support filtering. The raw label vocabulary contains 22,480 distinct surface forms, which is reduced to 19,096 canonical labels after normalization. Applying the support filter with $\tau = 5$ retains 5,771 labels and removes 13,325 low-support labels. The average number of labels per sample changes only slightly, from 3.6125 before filtering to 3.5662 after filtering, and only 27 samples have no retained label. This indicates that the support filter mainly stabilizes the label vocabulary while preserving macro-intent coverage for nearly all samples.

Table 1: Statistics of macro-intent labels after normalization and support filtering.

Statistic	Value
Number of samples	446,062
Raw label vocabulary size	22,480
Canonical label vocabulary size before filtering	19,096
Retained label vocabulary size ($\tau = 5$)	5,771
Removed low-support labels	13,325
Average labels per sample before filtering	3.6125
Average labels per sample after filtering	3.5662
Median labels per sample after filtering	3.0000
Samples with no retained label	27 (0.0061%)

Table 2 lists the most frequent retained macro-intent labels. Frequency denotes the percentage of samples containing each label. Because each sample may contain multiple labels, these frequencies are not mutually exclusive. The most frequent labels correspond to common algorithmic categories such as greedy methods, dynamic programming, graph theory, sorting, prefix sums, and string manipulation.

Table 2: Most frequent retained macro-intent labels. Frequency denotes the percentage of samples containing each label.

Label	Count	Frequency (%)
greedy	96,853	21.71
dynamic programming	77,879	17.46
math	66,289	14.86
simulation	61,060	13.69
combinatorics	54,773	12.28
graph theory	52,528	11.78
sorting	42,451	9.52
prefix sum	38,403	8.61
string manipulation	29,639	6.64
modular arithmetic	28,690	6.43
graph traversal	25,256	5.66
breadth first search bfs	24,916	5.59
binary search	22,717	5.09
sliding window	22,024	4.94
number theory	18,095	4.06

3 Selection Algorithm and Diagnostic Details

3.1 Selection Algorithm Details

All representations are ℓ_2-normalized before selection. For SAGC representations, we first concatenate the normalized micro-context vector and the normalized macro-intent vector, and then normalize the concatenated vector again. All diversity objectives use Euclidean distance in the corresponding representation space.

For distance-based objectives, we use greedy selection with a fixed random initialization. The first selected point is sampled using a fixed random seed, and the same initialization protocol is used across comparable runs. For Max-Sum, at each step we select the candidate whose total distance to the current selected set is largest:

$$x^* = \arg\max_{x \in \mathcal{D} \setminus S} \sum_{s \in S} d(x, s). \tag{1}$$

For Max-Min, at each step we select the candidate whose nearest distance to the selected set is largest:

$$x^* = \arg\max_{x \in \mathcal{D} \setminus S} \min_{s \in S} d(x, s). \tag{2}$$

To make greedy selection efficient, we maintain a length-N score array for all candidates. For Max-Sum, this array stores the accumulated distance from each candidate to the selected set and is updated after each newly selected point. For Max-Min, this array stores the current nearest distance from each candidate to the selected set and is updated by taking the elementwise minimum with the newly computed distance vector. Distance updates are vectorized and computed on GPU.

For K-Means selection, we use the `faiss` CPU implementation with $K = 256$ clusters for all representation spaces. Given a target subset size $B = 20{,}000$, we allocate samples as evenly as possible across clusters. If a cluster contains fewer samples than its allocated quota, the remaining sample slots are redistributed among the other clusters. Within each cluster, samples are selected randomly until the cluster quota is filled. The candidate pool is deduplicated during data construction, and each selection method returns a subset without duplicate samples.

3.2 Diagnostic Implementation Details

All embedding-space diagnostics are computed using Qwen3-Embedding-8B representations as the common reference space. Unless otherwise specified, we use the dense code embedding space for RI and RCD diagnostics. For Relative Isolation (RI), we use $k = 20$ nearest neighbors. For Relative Centroid Distance (RCD), the centroid is computed over the full candidate pool in the same dense reference space. The high-risk thresholds for RI and RCD are computed as $\mu_{\text{rand}} + 3\sigma_{\text{rand}}$ using a 20k random subset as the diagnostic reference. This diagnostic reference is separate from the five-seed average used for reporting random-sampling MCS. For each selected subset, r_{RI} and r_{RCD} denote the fraction of selected samples whose RI or RCD exceeds the corresponding threshold.

For surface rarity, we tokenize instructions using a regular-expression tokenizer that matches identifier-like strings, numbers, and individual non-whitespace symbols:

```
[A-Za-z_][A-Za-z_0-9]*|\d+|[^\s].
```
(3)

Token frequencies are computed over the instruction side of the full candidate pool. A token is considered rare if its full-pool frequency is no larger than $\eta = 5$. The rare instruction-token ratio R_{rare} is the average fraction of such rare tokens in the selected instructions.

For intent-distribution diagnostics, we use the filtered macro-intent label sets $\mathcal{T}'_x$. When a sample contains multiple labels, each retained label contributes once to the subset-level label count. Label entropy is computed over this pooled label-count distribution, and top-10 label mass is the fraction of all label occurrences accounted for by the ten most frequent labels in the selected subset.

For representation complementarity analysis, K-Means NMI is computed using $K = 256$ clusters for each representation space. The kNN overlap analysis uses $k = 20$ nearest neighbors. Both metrics are computed over the same candidate pool so that differences reflect the induced representation geometry rather than differences in sample coverage.

4 Experimental Configuration

All SFT experiments are conducted on an H200 GPU cluster using Qwen2.5-7B [3] as the base model. All selected subsets are trained under the same configuration so that performance differences are attributable to the selected data rather than training hyperparameters. Because no downstream validation signal is used during subset construction or training, we evaluate the final checkpoint for each run. Table 3 summarizes the SFT hyperparameters.

For *Random Sampling*, we train and evaluate five independently sampled 20k subsets and report the averaged MCS. For the other selection methods, we use the subset produced by the corresponding representation–objective configuration. Unless otherwise specified, LiveCodeBench MCS is averaged over 10 independent generation runs to mitigate decoding stochasticity.

Table 3: Hyperparameters for SFT.

Hyperparameter	Value
Global Batch Size	64
Sequence Length	32,768
Number of Epochs	4
Optimizer	AdamW
Learning Rate Schedule	Cosine
Peak Learning Rate	3×10^{-5}
Min Learning Rate	1×10^{-6}
Warmup Ratio	0.03

5 Benchmark Details

5.1 LiveCodeBench

LiveCodeBench [2] is used as our main evaluation benchmark. It consists of recent programming problems with executable test cases, making it suitable for temporally held-out code-generation evaluation. Following the main text, we evaluate on problems released after 2024-09-20 and report Mean Case Success (MCS), defined as the average fraction of passed test cases per problem. For the main LiveCodeBench results, MCS is averaged over 10 independent generation runs to reduce decoding stochasticity.

5.2 CodeEval-Pro

For out-of-benchmark evaluation, we use HumanEval-Pro and MBPP-Pro from CodeEval-Pro [4]. CodeEval-Pro constructs more challenging self-invoking code-generation problems based on the original HumanEval and MBPP tasks. Compared with direct function-level code generation, these benchmarks require the model to generate executable code that correctly invokes and composes the intended logic, thereby testing more complex instruction following and multi-step reasoning. In our evaluation, we report MCS for HumanEval-Pro and MBPP-Pro using a single generation run for each checkpoint.

5.3 Ag-LiveCodeBench-X

We also evaluate on the C++ split adapted from Ag-LiveCodeBench-X [1], a language-agnostic benchmark derived from LiveCodeBench. Ag-LiveCodeBench-X reformulates programming tasks into standard input/output problems, so that the same problem set can be evaluated under different programming-language configurations. Although the original Agnostics evaluation focuses on low-resource languages such as Lua, Julia, R, OCaml, and Fortran, the benchmark format itself is language-decoupled: each task is specified by a problem statement, standard-input test cases, and expected standard-output results. We therefore adapt the Ag-LiveCodeBench-X evaluation framework to C++ by configuring the compilation and execution commands for generated C++ solutions. This setting provides an additional out-of-benchmark test of whether the selected SFT data supports generalization to a widely used statically compiled language. As with CodeEval-Pro, we report MCS using a single generation run for each checkpoint.

6 Detailed Experimental Results

Table 4 provides the full set of evaluated generic fusion ablations and fused SAGC variants on LiveCodeBench. All methods select 20k samples from the same candidate pool, and results are reported in Mean Case Success (MCS), averaged over 10 independent generation runs. This table complements the compact main results table by reporting the representation–objective combinations that are not shown in the main text.

Table 4: Detailed LiveCodeBench results (MCS) for generic fusion ablations and fused SAGC variants under the 20k selection budget.

Representation	Diversity Objective		
	Max-Sum	K-Means	Max-Min
Naive Fusion Ablation			
Prompt (Qwen) & Code (Qwen)	0.3883	0.3831	0.3751
Prompt (Qwen) & Code (Jina)	0.3825	0.3845	0.3873
Prompt (Qwen) & Code (Qwen) & Label-Sum	0.3856	0.3847	0.3785
Prompt (Qwen) & Code (Qwen) & Label-Max	0.3666	0.3853	0.3771
Prompt (Qwen) & Code (Qwen) & Label-Weight	0.3739	0.3880	0.3804
Prompt (Qwen) & Code (Jina) & Label-Sum	0.3768	0.3731	0.3904
Prompt (Qwen) & Code (Jina) & Label-Max	0.3846	0.3857	0.3806
Prompt (Qwen) & Code (Jina) & Label-Weight	0.3753	0.3858	0.3943
Fused-Representation			
Prompt (Qwen) & Label-Sum	0.3771	0.3923	0.3765
Prompt (Qwen) & Label-Max	0.3768	**0.4006**	0.3980
Prompt (Qwen) & Label-Weight	0.3844	0.3773	0.3818
Code (Qwen) & Label-Sum	0.3823	0.3843	0.3801
Code (Qwen) & Label-Max	0.3727	0.3899	0.3848
Code (Qwen) & Label-Weight	0.3822	0.3900	0.3941
Code (Jina) & Label-Sum	0.3789	0.3888	0.3902
Code (Jina) & Label-Max	**0.3940**	0.3845	0.3807
Code (Jina) & Label-Weight	0.3856	0.3975	**0.4002**

References

[1] Aleksander Boruch-Gruszecki, Yangtian Zi, Zixuan Wu, Tejas Oberoi, Carolyn Jane Anderson, Joydeep Biswas, and Arjun Guha. 2025. Agnostics: Learning to code in any programming language via reinforcement with a universal learning environment. *arXiv preprint arXiv:2508.04865* (2025).

[2] Naman Jain, King Han, Wen-Ding Gu, Alex andJD, Fanjia Yan, Pin-Yu Tian, Hao Wang, and Xin Strong, Chiang andXie. 2024. LiveCodeBench: Holistic Evaluation of LLMs for Code Programming. *arXiv preprint arXiv:2403.07974* (2024). https://arxiv.org/abs/2403.07974

[3] An Yang, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoyan Huang, Jiandong Jiang, Jianhong Tu, Jianwei Zhang, Jingren Zhou, et al. 2025. Qwen2. 5-1m technical report. *arXiv preprint arXiv:2501.15383* (2025).

[4] Zhaojian Yu, Yilun Zhao, Arman Cohan, and Xiao-Ping Zhang. 2025. HumanEval pro and MBPP pro: Evaluating large language models on self-invoking code generation task. In *Findings of the Association for Computational Linguistics: ACL 2025*. 13253–13279.